



UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
SISTEMAS OPERACIONAIS

ATIVIDADE 1

FERNANDA KAROLINY SANTOS SILVA

PROF. DRº GLAUCO DE FIGUEIREDO CARNEIRO

São Cristóvão - SE

2026

SUMÁRIO

1. Introdução.....	3
2. Fundamentação Teórica.....	3
3. Problema.....	4
4. Informações.....	4
4.1. Ambiente.....	4
4.2. Camada de Aplicação.....	4
4.3. Trilha.....	5
4.4. Modelo.....	5
5. Relação do Sistema Operacional com IA Generativa.....	7
6. Resultados.....	7
6.1. Processos e Threads.....	7
6.2. Chamadas de Sistema.....	8
7. URL do Repositório e Vídeo.....	11
8. Uso de IA Generativa.....	11
9. Conclusão.....	12

1. Introdução

Aplicações de Inteligência Artificial (IA) generativa executadas localmente mobilizam uma infraestrutura complexa de processos, *threads*, memória e comunicação em rede. Nesse contexto, o sistema operacional (SO) atua como um mediador essencial, fornecendo abstrações que tornam o hardware utilizável e controlável. Este relatório documenta a instalação, configuração e observação de um sistema baseado no motor Ollama. O problema central investigado é como a camada de aplicação, o modelo de linguagem quantizado e as configurações de concorrência afetam a criação de processos, o consumo de recursos (CPU, RAM e armazenamento) e a responsividade geral do SO.

2. Fundamentação Teórica

O sistema operacional não é um bloco único, sendo constituído por componentes como o núcleo (kernel), código de inicialização, drivers e utilitários. O núcleo executa em modo privilegiado e gerencia tarefas, memória e arquivos, garantindo isolamento através da Unidade de Gerência de Memória (MMU).

Para que aplicações em nível de usuário interajam com o hardware, elas utilizam chamadas de sistema (*syscalls*), que disparam interrupções de software (*traps*) transferindo, de forma controlada, o controle ao núcleo. O trabalho do processador é dividido em tarefas, que são conceitos dinâmicos com estado interno em constante evolução, podendo ser implementadas como processos ou *threads*.

Um processo atua como um contêiner de recursos, englobando áreas de código, dados, pilha e descritores de arquivos, o que garante o isolamento da execução. Já as *threads* são fluxos de execução contidos dentro de um processo. Sistemas operacionais modernos geralmente adotam a abordagem 1:1, na qual a gerência de *threads* é incorporada ao núcleo e cada *thread* de usuário é mapeada para uma *thread* de núcleo, viabilizando o uso eficiente de múltiplos processadores simultaneamente.

Além disso, o SO adota o modelo de preempção por tempo (tempo compartilhado), garantindo que cada tarefa receba uma fatia de processamento (*quantum*) da CPU, evitando que aplicações entrem em laços infinitos e monopolizem o sistema.

3. Problema

O desafio técnico consiste em monitorar e diagnosticar o impacto sistêmico da inferência de um modelo de linguagem local, mapeando os processos criados, as chamadas de sistema executadas e os gargalos de recursos, como a competição por CPU e a paginação por falta de espaço em memória.

4. Informações

4.1. Ambiente

Abaixo constam as características do computador utilizado para a execução do projeto:

- **Sistema Operacional/Distribuição:** Fedora Linux
- **Versão:** 44 Workstation Edition
- **Interface Gráfica:** GNOME
- **Tipo de Ambiente:** Nativo
- **Kernel:** 7.1.13-200.fc44.x86_64
- **CPU:** Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz
- **RAM:** 8GB
- **Armazenamento:** SSD 500G
- **GPU:** Intel Corporation WhiskeyLake-U GT2 [UHD Graphics 620]
- **Núcleos/Threads:** 4 núcleos (Quad-Core) e 8 threads
- **Versão Ollama:** 0.34.0
- **Dependências da Camada de Aplicação:**

O levantamento do inventário do ambiente foi realizado via terminal utilizando os seguintes comandos do sistema:

- `cat /etc/os-release`
- `echo $XDG_CURRENT_DESKTOP`
- `uname -r`
- `lscpu`
- `free -h`
- `df -h`
- `lspci | grep -E "VGA|3D"`
- `ollama --version`

4.2. Camada de Aplicação

Nome: Open WebUI

URL do Repositório: <https://github.com/open-webui/open-webui>

Licença: Open WebUI License. Trata-se de uma licença permissiva de código aberto, fortemente baseada na *BSD 3-Clause*, que autoriza o uso, a modificação e a distribuição do *software* (em código-fonte ou binário). O seu

principal diferencial técnico é a adição de uma cláusula restritiva de proteção de marca (Cláusula 4): é estritamente proibido remover, ocultar ou alterar a identidade visual e o nome "Open WebUI" em ambientes que possuam mais de 50 usuários simultâneos, a menos que seja adquirida uma licença corporativa (*enterprise*) específica.

Data de Acesso: 15/09/2026

Finalidade da aplicação e arquitetura simplificada:

Funciona como uma interface *web* rica e orquestradora para Modelos de Linguagem Grande (LLMs). Nesta execução unificada (*bundled*), tanto o *frontend/backend* (Open WebUI) quanto o *runtime* de inferência (Ollama) estão encapsulados no mesmo container Docker, rodando como processos independentes geridos pelo *daemon* do contêiner.

Dependências e comandos de instalação e execução:

```
docker run -d -p 3000:8080 -v ollama:/root/.ollama -v
open-webui:/app/backend/data --name open-webui --restart always
ghcr.io/open-webui/open-webui:ollama
```

Adaptações realizadas, limitações e erros encontrados:

Devido à presença de uma GPU integrada (Intel UHD 620), optou-se pela imagem focada em execução por CPU.

4.3. Trilha

Trilha A - Chat local: Ollama + Open WebUI:

A Trilha A permite uma análise rigorosa do tráfego de rede local (sockets e portas) e da execução containerizada. A escolha por uma interface baseada em tecnologias modernas de front-end, como React, Next.js e TypeScript, facilita o rastreamento da comunicação HTTP entre o cliente e a API local do Ollama, demonstrando como o SO gerencia múltiplas requisições concorrentes e o redirecionamento de entrada/saída.

4.4. Modelo

Identificação:

- **Organização:** Microsoft
- **Nome:** Phi-3.5-mini-instruct
- **URL:** <https://huggingface.co/microsoft/Phi-3.5-mini-instruct>

Parâmetros: 3,82 bilhões de parâmetros (3.8B).

Finalidade e idioma: Modelo ajustado para instruções (instruct/chat), com foco forte em lógica, raciocínio e código. É multilíngue e possui suporte ao idioma português, embora tenha desempenho superior em inglês.

Licença: MIT License. É uma licença altamente permissiva e open-source. Permite uso comercial, modificação e distribuição, sendo a única obrigação a manutenção do aviso original de direitos autorais.

Formato e quantização: O modelo base na URL oficial está em Safetensors (formato BF16). Para execução local via Ollama, ele é convertido e baixado no formato GGUF (geralmente sob quantização Q4_0 ou Q4_K_M para otimizar espaço).

Contexto: Janela máxima de 128.000 tokens (128k). A limitação na prática dependerá da quantidade de memória RAM disponível na sua máquina para processar esse limite total.

Arquivos e hardware:

- **Tamanho:** O arquivo quantizado (Ollama) pesa cerca de 2,2 GB.
- **RAM/VRAM:** Exige aproximadamente 4 GB a 6 GB de memória livre.
- **CPU/GPU:** Altamente viável localmente, rodando de forma fluida até mesmo em CPUs modernas sem necessidade de uma GPU dedicada.

Riscos e limitações: Confabulação/Viés: Como qualquer LLM, o modelo está sujeito a alucinações (inventar fatos de forma confiante) e pode carregar vieses sociais dos dados de treino.

Segurança: É alinhado para evitar gerar conteúdos nocivos, mas não deve ser usado para conselhos críticos (médicos, legais).

Privacidade: Excelente, pois ao rodar via Ollama no seu hardware, os dados inseridos no prompt não são enviados para nenhum servidor externo.

```
karoliny@fedora:~$ time docker exec -it open-webui ollama pull phi3.5:3.8b-mini-instruct-q4_0
pulling manifest
pulling b5374915da53: 100% | 2.2 GB/2.2 GB 31 MB/s 0s
verifying sha256 digest
writing manifest
success

real    1m28,767s
user    0m0,080s
sys     0m0,102s
```

Figura 1

Como ilustrado na Figura 1, o download do modelo possui o tamanho de 2,2 GB e foi concluído em 1 minuto e 28 segundos.

A análise do tempo de download do modelo, registrado via utilitário `time`, comprovou na prática a teoria de estados de processos. O tempo real (relógio) foi substancialmente maior que a soma do tempo de usuário (`user`) e de sistema (`sys`). Isso ocorre pois o processo passou a maior parte do tempo no estado suspenso/bloqueado, aguardando interrupções de hardware da placa de rede para o recebimento dos pacotes I/O do download, não consumindo ciclos de CPU ativos neste período.

5. Relação do Sistema Operacional com IA Generativa

A execução de um modelo de Inteligência Artificial depende de uma infraestrutura de hardware cujo acesso é estritamente orquestrado e abstraído pelo Sistema Operacional (SO). A aplicação web (Open WebUI) comunica-se com a API do motor (Ollama) via soquetes de rede local, enquanto as bibliotecas de baixo nível do motor interagem diretamente com a Interface Binária de Aplicação (ABI) para viabilizar os complexos cálculos matriciais.

Para realizar a inferência, o processo emite chamadas de sistema (*syscalls*) que delegam tarefas críticas ao *kernel*. Isso inclui mapear os massivos arquivos de pesos para a memória virtual (paginação), sincronizar múltiplas *threads* para explorar o paralelismo da CPU e gerenciar requisições assíncronas, provando que o SO atua como o mediador fundamental entre o modelo de linguagem e o hardware físico.

6. Resultados

6.1. Processos e Threads

Ao rodar o comando `ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu` e analisar o retorno concluiu-se as informações abaixo.

Identificação e Hierarquia (PID e PPID):

- Gerenciador (Pai de todos): `containerd-shim` (PID 59220) – Responsável por manter o contêiner do Docker ativo no sistema operacional hospedeiro.
- Camada de Aplicação (Processo Auxiliar): `python3 / Open WebUI` (PID 59247). Seu PPID é 59220, confirmando que foi instanciado pelo Docker.

- Motor de Inferência (Processo Principal): ollama (PID 59311). Seu PPID é 59247, provando que é a camada Python quem aciona e controla o ciclo de vida do motor da IA.

Estados dos Processos (STAT):

- Ambos os processos principais apresentam a flag **SI**.
- O **S** (**Interruptible Sleep**) indica que, no momento da medição, os processos estavam bloqueados/suspensos na CPU aguardando interrupções (como mensagens de rede do chat).
- O **I** (**multi-threaded**) sinaliza ao *kernel* que essas tarefas não são sequenciais simples, mas ramificadas em várias *threads*.

Quantidade de Threads (NLWP - *Number of Light Weight Processes*):

- **Aplicação (python3)**: Possui 36 *threads* abertas para gerenciar conexões assíncronas HTTP da interface web.
- **Ollama (ollama)**: Possui 24 *threads* de prontidão, alocadas para paralelizar ao máximo os cálculos matriciais dos pesos (Phi-3.5) nos 8 núcleos lógicos do processador.

Consumo de Recursos (%CPU e %MEM):

Em estado de ociosidade (antes de iniciar a inferência):

- **Ollama**: Consumia 7,8% da CPU e 0,2% da RAM.
- **Aplicação (Python)**: Consumia 3,4% da CPU e 0,4% da RAM.

Relação entre os Componentes:

- O isolamento de todo o ecossistema é garantido pelo Docker. A camada de aplicação atua como a interface de comunicação (porta 3000 mapeada para 8080), recebendo os *prompts* do usuário e despachando as solicitações via chamadas internas (socket) para o motor do Ollama, que processa a LLM e devolve a resposta.

6.2. Chamadas de Sistema

Ao rodar o comando “`strace -f -c -p 59311 -o strace-resumo.txt`” e analisar o retorno concluiu-se as informações abaixo.

Durante a execução controlada com a ferramenta *strace* acoplada ao PID do motor de inferência (Ollama), observou-se um massivo *overhead* no sistema. Devido à natureza altamente paralelizada da aplicação (24 *threads* ativas no processador) somada à interceptação simultânea de milhares de chamadas em nível de núcleo pelo *strace*, o sistema operacional esgotou os recursos

físicos de RAM e CPU, evidenciando o alto custo de mudança de contexto (*context switch*) imposto por ferramentas de depuração em cargas de I.A.

```
karoliny@fedora:~$ sudo strace -f -c -p 59311 -o strace-resumo.txt
[sudo] senha para karoliny:
strace: Process 59311 attached with 24 threads
strace: Process 66024 attached
strace: Process 66025 attached
strace: Process 66026 attached
strace: Process 66027 attached
strace: Process 66028 attached
strace: Process 66029 attached
strace: Process 66030 attached
strace: Process 66031 attached
strace: Process 66032 attached
strace: Process 66033 attached
strace: Process 66034 attached
strace: Process 66035 attached
strace: Process 66083 attached
strace: Process 66084 attached
strace: Process 66085 attached
strace: Process 66152 attached
```

Figura 2

Evidência de Alocação Dinâmica de Threads:

Conforme ilustrado na captura de tela da execução do utilitário `strace` (utilizando a flag `-f` para rastreamento de subprocessos) na Figura 2, foi possível observar empiricamente o gerenciamento de concorrência em tempo real. A ferramenta confirmou a alocação inicial de 24 *threads* de prontidão no processo principal do motor de inferência (corroborando a métrica NLWP anterior) e capturou a instanciação dinâmica de novas *threads* (ex: PIDs 66024, 66025, etc.) no exato instante em que a requisição de inferência foi despachada. Este comportamento evidencia a escalabilidade da aplicação e a eficiência do *kernel* do Linux na criação de *Light Weight Processes* para distribuir a carga matricial pelos múltiplos núcleos do processador.

Análise das Chamadas de Sistema (Syscalls):

O monitoramento ocorreu durante o envio de uma requisição pelo modelo Phi-3.5-mini-instruct. Embora a requisição tenha sido interrompida rapidamente pela camada de aplicação devido a uma incompatibilidade de API (a interface tentou acionar recursos externos de *Tools/Web Search*, o que não possui suporte nativo na versão quantizada do modelo), o `strace` capturou com sucesso o comportamento do *kernel* na recepção assíncrona, no despertar de *threads* e no tratamento de erro interno. A tabela de

resultados gerada evidenciou que o processo realizou 38.726 chamadas de sistema no total.

Com base nesta execução, destacam-se três chamadas (ou famílias) fundamentais:

- **futex (Sincronização de Threads - 80,36% do tempo total)**

Chamadas registradas: 13.927

Análise: O futex (*Fast Userspace Mutex*) dominou o perfil de execução. O alto volume de chamadas comprova a contenção intensa gerada pela necessidade de sincronizar a memória compartilhada entre as dezenas de *threads* ativas. O *kernel* atua arbitrando o acesso aos tensores do modelo para evitar condições de corrida (*race conditions*) durante as operações, mantendo o controle de concorrência.

- **epoll_pwait (Multiplexação de I/O de Rede - 6,72% do tempo total)**

Chamadas registradas: 1.829

Análise: Essencial para o fluxo de dados assíncrono entre a camada de aplicação web (Open WebUI) e o motor Ollama. O *epoll_pwait* permite que o processo durma de forma eficiente na CPU e seja acordado pelo *kernel* apenas quando há dados prontos no soquete (como a chegada da requisição ou o envio do erro retornado). Seu uso demonstra uma arquitetura de rede não-bloqueante (*non-blocking I/O*) altamente otimizada.

- **mmap e munmap (Gerenciamento de Memória)**

Chamadas registradas: 167 (mmap) e 27 (munmap)

Análise: Em vez de realizar chamadas sequenciais de leitura (read) no disco para processar o modelo, o processo utiliza chamadas *mmap* para mapear os arquivos de pesos (.gguf) diretamente para o espaço de endereçamento virtual da memória RAM. Isso delega a paginação sob demanda para a unidade de gerenciamento de memória (MMU) do *kernel*, permitindo acesso rápido aos blocos de dados.

```
karoliny@fedora:~$ cat strace-resumo.txt
```

% time	seconds	usecs/call	calls	errors	syscall
80,36	747,932651	53703	13927	9403	futex
6,93	64,520519	64520518	1		waitid
6,72	62,516762	34180	1829	2	epoll_pwait

Figura 3

0,00	0,001747	10	167		mmap
0,00	0,001743	12	140		getdents64
0,00	0,001668	11	142		madvise
0,00	0,001478	25	57		mprotect
0,00	0,001468	6	225	30	stat
0,00	0,001378	8	170	11	newfstatat
0,00	0,001113	11	93		getsockopt
0,00	0,000956	18	51		rt_sigprocmask
0,00	0,000773	2	325		fcntl
0,00	0,000590	36	16		clone
0,00	0,000538	19	27		munmap

Figura 4

As figuras 3 e 4 mostram algumas chamadas de sistema executadas e seus números de chamadas.

7. URL do Repositório e Vídeo

Segue abaixo os links para o repositório no GitHub e para o vídeo de apresentação.

Repositório do GitHub:

https://github.com/FernandaKaroliny/SO_UFS_2026_2_Karoliny_Fernanda

URL do Vídeo:

 2026.2 - SO - Atividade 1

8. Uso de IA Generativa

O assistente foi utilizado como ferramenta de apoio para a interpretação dos requisitos da atividade, compreensão e construção dos conceitos teóricos da disciplina de Sistemas Operacionais, correção ortográfica e de coesão textual, obtenção de comandos Linux para levantamento de características de hardware e software do notebook, auxílio técnico na escolha do modelo

LLM mais adequado para o hardware disponível (limitado a 8GB de RAM e sem GPU dedicada), e suporte na interpretação técnica dos *logs* gerados pelas ferramentas ps e strace.

Ferramenta e modelo utilizados: Google Gemini 3.1 Pro

Testes e observações usadas para verificar respostas: As respostas da IA não foram assumidas como verdade absoluta. Toda a análise foi validada através da observação direta do comportamento do terminal no Fedora Linux e do cruzamento de dados de PIDs, *syscalls*.

9. Conclusão

A atividade teve como objetivo ilustrar como uma aplicação de IA generativa tensiona as estruturas de um sistema operacional moderno. Com a execução da Trilha A, comprovou-se que o desempenho da IA depende da eficiência do escalonador do *kernel* em distribuir as *threads* e gerenciar a sincronização (evidenciado pela alta taxa de chamadas *futex*). Paralelamente, o uso intenso de *mmap* demonstrou a elegância da gestão de memória virtual, delegando a paginação sob demanda à MMU (*Memory Management Unit*) e otimizando o I/O no hardware sem sobrecarregar a RAM.

Por fim, o esgotamento de recursos ocorrido durante o monitoramento com o *strace* ilustrou, na prática, o altíssimo custo do *overhead* de mudança de contexto (*context switch*). Conclui-se que, embora arquiteturas em contêineres como o Docker garantam um excelente isolamento de dependências, a disputa pelos ciclos de CPU e pela tabela de memória continua sendo arbitrada implacavelmente pelo *kernel* subjacente.